Esquema de Trabajo: Localización de Fuentes de PM10

Problema Inverso de Dispersión

El problema

El proyecto aborda la problemática crítica de la dispersión de material particulado PM10 en el Valle de Aburrá, donde la geografía irregular y las condiciones meteorológicas variables dificultan la identificación precisa de las fuentes de emisión. Para resolver este problema inverso, se propone el uso de *Adaptive Inverse PINNs* (Redes Neuronales Informadas por la Física), las cuales integran datos del ecosistema SIATA con la ecuación matemática de advección-difusión-reacción para modelar el transporte y la degradación de partículas. Esta metodología utiliza un mecanismo de pérdida adaptativa con pesos dinámicos y una arquitectura agéntica para superar las limitaciones de datos y la naturaleza *mal planteada* (*ill-posed*) del sistema físico, permitiendo localizar "hotspots" de contaminación y diferenciar entre fuentes industriales y de tráfico de manera precisa.

Inverse PINN Adaptativa

PINN Inversa Estándar

Una *Physics-Informed Neural Network* (PINN) inversa tiene como objetivo estimar parámetros constitutivos θ_0 (coeficientes físicos, velocidades, etc.) que aparecen en la EDP subyacente, a partir de datos observados. El problema de minimización toma la forma

$$\left(\theta^{\dagger},\, \theta_0^{\dagger}\right) = \arg\min_{\theta,\,\theta_0} \mathcal{L}\big(u_{\mathrm{NN}}(\theta);\, \theta_0\big) + \iota\, \|\theta_0 - \theta_0^{*}\|^2, \tag{1}$$

donde $\mathcal{L}$ combina la pérdida de ajuste a datos y la pérdida residual de la EDP, ι es un parámetro de regularización e θ_0^{*} son parámetros de referencia. En ausencia de regularización se toma $\iota = 0$.

El principal inconveniente de esta formulación es que los distintos términos de $\mathcal{L}$

pueden entrar en conflicto durante la retropropagación, produciendo gradientes desequilibrados y dificultando la convergencia, problema que se agrava cuando los datos disponibles son escasos [**?**].

PINN Inversa Adaptativa

Para superar la inestabilidad de la PINN inversa estándar, Berardi et al., 2025, proponen una **función de pérdida ponderada adaptativa**:

$$\mathcal{L}\big(u_{\mathrm{NN}}(\theta);\,\theta_0\big) = \sum_{i=1}^{M}\Big(\omega_i^k\,\|u(x_i^*,t_i^*) - u_i^*\|^2 + \omega_{\mathcal{R}}^k\,\|\mathcal{R}(u(x_i^*,t_i^*);\theta_0)\|^2\Big), \tag{2}$$

donde los pesos ω_i^k y $\omega_{\mathcal{R}}^k$ se actualizan en cada época k de acuerdo con

$$\omega_i^k = \frac{\hat{\omega}_i^k}{\displaystyle\sum_{j=1}^{M}\hat{\omega}_j^k + \hat{\omega}_{\mathcal{R}}^k}, \qquad \omega_{\mathcal{R}}^k = \frac{\hat{\omega}_{\mathcal{R}}^k}{\displaystyle\sum_{j=1}^{M}\hat{\omega}_j^k + \hat{\omega}_{\mathcal{R}}^k}, \tag{3}$$

y los pesos crudos $\hat{\omega}_i^k$ se asignan según la naturaleza del punto de entrenamiento:

$$\hat{\omega}_i^k = \begin{cases} \eta_{\mathrm{BC}} & \text{si } x_i^* \in \partial\Omega, \\ \eta_{\mathrm{IC}} & \text{si } t_i^* = 0, \\ \nu(k)\,\eta_u & \text{si } (x_i^*, t_i^*) \text{ es punto de colocación}, \\ 0 & \text{en otro caso.} \end{cases} \tag{4}$$

La función $\nu(k)$ es creciente con la época k, con $\nu(0) = 0$ y $\nu(k) \to 1$ cuando $k \to \infty$. En concreto se adopta

$$\nu(k) = \frac{\tanh\!\left(10\,\dfrac{k - K/2 - K_0}{K}\right) + 1}{2}, \qquad k = 1, \ldots, K, \tag{5}$$

donde K es el número total de épocas y K_0 es un umbral a partir del cual los pesos comienzan a modificarse de forma significativa.

Diferencia radical con la PINN inversa estándar. La novedad fundamental reside en el *curriculum de entrenamiento* que introduce $\nu(k)$:

- **Fase inicial ($k \ll K/2$):** $\nu(k) \approx 0$, por lo que los puntos de colocación reciben peso nulo. La red se entrena *únicamente* con el residuo de la EDP, aprendiendo

primero la dinámica física sin interferencia de los datos ruidosos.

- **Fase tardía ($k \gg K/2$):** $\nu(k) \to 1$, los datos observados se incorporan progresivamente a la pérdida y los parámetros físicos θ_0 comienzan a actualizarse de forma efectiva, pues sus gradientes se escalan por $\gamma\,\nu(k)$.

Este mecanismo evita que los gradientes de los parámetros físicos corrompan la solución antes de que la red haya aprendido la física del problema, garantizando la convergencia incluso desde condiciones iniciales aleatorias. La tasa de aprendizaje se actualiza además con una estrategia de decaimiento exponencial por pasos:

$$\alpha_k = \alpha_0\,\beta^{\lfloor k/100 \rfloor}, \qquad 0{,}9 < \beta < 0{,}99. \tag{6}$$

El proceso completo se resume en el Algoritmo 1.

Algorithm 1 Entrenamiento con pesos adaptativos y actualización de gradientes

```
epoch ← 0
repeat
    epoch ← epoch + 1
    if do_parameter_train and epoch > K_0 then
        Calcular ν(epoch) según Ec. (5)
    end if
    Actualizar pesos de datos según Ec. (4)
    Calcular gradientes de L
    Reescalar ∇_{θ_0} L por ν(epoch)
    Aplicar gradientes a todos los parámetros entrenables
until convergencia o epoch > K
```

1. Fundamento Arquitectónico

La estrategia se basa en disociar la **orquestación semántica** (configuración, supervisión y validación) de la **solución de la EDP** (cómputo numérico). Los *Large Language Models* (LLMs) se limitarán al rol de Agentes, y no al cálculo directo de ecuaciones diferenciales. El objetivo es minimizar la carga computacional y maximizar la precisión en la identificación de fuentes $S(\vec{p},\, t)$.

1.1. Adquisición y Preprocesamiento de Datos

La alimentación de la función de pérdida empírica $\mathcal{L}_{\text{datos}}$ exige la integración automatizada de datos de alta resolución espacial.

Cuadro 1: Fuentes de datos, herramientas y preprocesamiento.

Fuente de Datos	**Variables Clave**	**Herramientas / Estrategia**	**Preprocesamiento Crucial**
Red SIATA (Monitoreo)	Concentraciones de PM10 y datos meteorológicos (frecuencia de 10 minutos).	Utilizar Socrata API (`sodapy`) o peticiones al portal CKAN del Área Metropolitana, dada la no disponibilidad de una API REST pública de SIATA para descargas masivas históricas. Indagar en la página web de Calidad Aire: `https://siata.gov.co/CalidadAire/`	**Adimensionalización:** Escalar el dominio espacio-temporal (Valle de Aburrá y el tiempo) a $[-1, 1]^3$ y $[0, 1]$ respectivamente, para mitigar el problema de gradientes patológicos en las *Physics-Informed Neural Networks* (PINN).
Datos Satelitales (AOD) *(Opcional)*	*Aerosol Optical Depth* (AOD).	Google Earth Engine (GEE) Python API para extraer series temporales reproyectadas y enmascaradas (Sentinel-5P, MODIS, VIIRS), evitando la descarga de terabytes de imágenes crudas.	**Collocation Points Inteligentes:** Implementar *Latin Hypercube Sampling* (LHS). Priorizar la densidad de puntos cerca de estaciones de monitoreo y zonas de topografía compleja en lugar de utilizar una malla regular.

2. Ecosistema de Modelado (Solver de la EDP)

El motor computacional debe ser eficiente para resolver la ecuación de Advección-Difusión-Reacción (ADR) y la optimización simultánea:

$$\arg\min_{\theta,\,\theta_0} \mathcal{L}(u_{NN}(\theta);\, \theta_0)$$

Cuadro 2: Componentes del ecosistema de modelado.

Componente	**Descripción**	**Estrategia / Frameworks**
Motor de Cómputo	Solución de la EDP y cálculo de la función de pérdida.	**Opción 1 (Alto Rendimiento):** Ecosistema Julia (`NeuralPDE.jl`, `Lux.jl`) para compilar la EDP a código máquina y reducir el costo. **Opción 2 (Python):** NVIDIA Modulus o DeepXDE (maduros para problemas inversos y soporte de reponderación adaptativa de pérdida).
Parametrización de Fuentes	Representación eficiente de la fuente $S(\vec{p}, t)$.	Parametrizar como una **suma de distribuciones Gaussianas bidimensionales**. La PINN inversa aprende las coordenadas del centroide (μ_x, μ_y) y la amplitud, lo que suaviza el panorama de optimización espacial.

3. Arquitectura Agéntica (Flujo de Trabajo MLOps)

Se propone la implementación de cinco agentes utilizando frameworks *open-source* como CrewAI o Microsoft AutoGen.

4. Propuesta de Ejecución Escalonada (*Curriculum Learning*)

Para garantizar la estabilidad y la convergencia del entrenamiento se proponen tres fases progresivas:

1. **Fase Interpolativa.** Entrenar una red *forward* estándar solo con datos de SIATA, asumiendo fuentes $S = 0$, para aprender el campo de vientos y concentraciones.
2. **Fase Inversa Estática.** Congelar los pesos de la red anterior. Iniciar la Inverse PINN, añadir el término $S(\vec{p}, t)$ y entrenar **exclusivamente** para descubrir las coordenadas de las fuentes y el parámetro de difusión D.
3. **Ajuste Fino Adaptativo.** Aplicar una metodología (e.g., Berardi et al.) para un afinamiento conjunto de **todos** los parámetros, permitiendo que la física guíe el aprendizaje.

Cuadro 3: Agentes del sistema y sus funcionalidades.

Agente	**Rol Principal**	**Funcionalidad Clave**
Agente 1: Physics Architect	El Configurador	Define los límites del dominio y establece las condiciones de frontera (Dirichlet / Neumann) basándose en datos de SIATA y GEE. Genera el script de configuración inicial de la PINN.
Agente 2: PINN-ADR Solver	El Ejecutor (*Tool*)	**No es un LLM.** Es un script empaquetado (Python / Julia) que el framework agéntico invoca. Entrena la red, recibe parámetros y devuelve el *loss* y los gradientes.
Agente 3: Reaction Validator	El Validador Científico	Accede a literatura (vía RAG) para verificar que las tasas de decaimiento k descubiertas por la Inverse PINN son consistentes con la termodinámica y cinética del PM10.
Agente 4: Source Identification	El Atribuidor Forense	Recibe las coordenadas optimizadas $\vec{p}$ y realiza un cruce espacial (GeoPandas / OpenStreetMap API) para atribuir la fuente a zonas industriales, corredores de tráfico o factores topográficos.

Justificación de la Fase Interpolativa: Pre-acondicionamiento para la Inverse-PINN

Atacar el problema de la identificación de parámetros físicos directamente con una Inverse-PINN (que aprende simultáneamente el campo de concentración y los parámetros desconocidos) es una estrategia teóricamente válida, pero altamente inestable y computacionalmente ineficiente si se comienza con una inicialización aleatoria de pesos. El principal desafío radica en la naturaleza de "problema mal-puesto" (*ill-posed*) debido a la escasez de datos, lo que provoca un conflicto en las contribuciones de las funciones de pérdida.

La fase interpolativa se introduce como un mecanismo de **pre-acondicionamiento** esencial del espacio de pesos para mitigar estos riesgos:

Cuadro 4: Problemas de la Inverse-PINN directa y sus soluciones mediante pre-acondicionamiento.

Problema con la Inverse-PINN Inicial	**Solución a través del Pre-acondicionamiento**
Optimización No Convexa Extrema	La red debe simultáneamente identificar la topología del campo escalar de concentración $C(\theta)$ y los parámetros físicos θ_0 (difusión, coordenadas de fuente), resultando en un panorama de optimización plagado de mínimos locales.
Gradientes Dominantes y Patológicos	Al inicio, el residuo físico (término $\mathcal{L}_{EDP}$) es desproporcionadamente grande. Los gradientes de $\mathcal{L}_{EDP}$ anulan a $\mathcal{L}_{\text{datos}}$, forzando a la red a soluciones triviales que satisfacen la matemática de la EDP (e.g., concentración nula), pero ignoran los datos reales del SIATA.
Alta Susceptibilidad al Ruido y Sobreajuste	La red es altamente sensible a la discretización y el ruido inherente de los datos puntuales de las estaciones SIATA. Esto puede llevar a que la red "sobreajuste" los parámetros físicos (e.g., alterar artificialmente el coeficiente de difusión D) solo para justificar picos anómalos o ruido en las lecturas, comprometiendo la fidelidad de la física subyacente.

Función de Pérdida Global

La función de pérdida total es:

$$\mathcal{L}(u(\theta)) = \mathcal{L}_{\text{datos}}(\theta) + \mathcal{L}_{EDP}(\theta).$$

La fase interpolativa busca primero minimizar $\mathcal{L}_{\text{datos}}$ para establecer una base de campo escalar físicamente plausible antes de introducir la penalización estricta de $\mathcal{L}_{EDP}$ para la identificación de θ_0.